Ask The Docs

Build & Deploy a RAG Chatbot

From Zero to a Live AWS-Hosted AI Application

100% Open Source • Zero Paid APIs • Dockerized • AWS EC2 Deployed

A complete step-by-step guide for developers

01 What Is RAG? (The Big Idea)

RAG stands for Retrieval Augmented Generation. It is the technique that powers modern AI assistants that can answer questions about YOUR specific documents — not just general knowledge.

The Open-Book Exam Analogy

Think of a regular AI as a student taking a closed-book exam. It can only answer from memory. RAG is like giving that student the textbook during the exam. It finds the right page, reads it, and answers accurately.

How RAG Works in 5 Steps

1. User uploads a document (PDF or TXT)
▼
2. Document is split into small chunks of text
▼
3. Each chunk is converted into a number-vector (embedding)
▼
4. Vectors are stored in a searchable database (FAISS)
▼
5. User asks a question → system finds the closest chunks → AI generates an answer

💡 Why Not Just Feed the Whole Document to the AI?

Most AI models have a token limit (e.g., 512 or 4096 tokens). A 50-page PDF far exceeds this. RAG solves this by retrieving only the relevant 2–3 chunks the model actually needs.

02 Tech Stack — 100% Free & Open Source

Every single tool in this project is free. No credit card needed for the AI models. No API keys. Everything runs locally on your own hardware.

Component	Tool / Library	Why This Choice
UI	`Streamlit`	Python-native web framework. Build interactive apps in minutes — no frontend coding needed.
PDF Reader	`PyPDF2 + pdfplumber`	Extracts text from normal PDFs. pdfplumber handles tables better.
OCR Engine	`Tesseract + pdf2image`	Reads scanned (image-based) PDFs. Converts page images into searchable text.
Chunking	`LangChain`	Splits text into overlapping chunks so no context is lost at boundaries.
Embeddings	`all-MiniLM-L6-v2`	Converts text → 384-dim vectors. Only 80 MB. Fast on CPU.
Vector DB	`FAISS`	Facebook's similarity search library. Stores & searches vectors instantly. No server needed.
LLM	`google/flan-t5-base`	250M parameter model. Runs on CPU. Answers questions from context. 850 MB total.
Container	`Docker`	Packages the entire app so it runs identically anywhere — your laptop or AWS.
Cloud	`AWS EC2`	Rents a virtual machine in the cloud. App is publicly accessible via URL.

⚡ CPU & Memory Reality Check

The embedding model uses ~200 MB RAM. The LLM uses ~1.5 GB. Total app footprint is around 2.5–3 GB. A t2.medium instance (4 GB RAM) is the sweet spot. t2.micro (1 GB) will crash.

03 Project Folder Structure

This is the exact file layout you need. Every file has one clear job.

```
ask-the-docs/
│
├── app.py                    ← Main Streamlit UI (ties everything together)
├── requirements.txt          ← Python library list
├── Dockerfile                ← Docker build instructions
├── .dockerignore             ← Files Docker should skip
├── README.md                 ← Project documentation
│
├── utils/
│   ├── __init__.py           ← Makes 'utils' a Python package
│   ├── document_loader.py    ← Reads PDF/TXT, runs OCR if needed
│   ├── text_processor.py     ← Splits text into chunks
│   ├── embeddings.py         ← Creates vectors + builds FAISS index
│   └── llm_handler.py        ← Sends question + context to LLM
│
├── data/uploads/             ← Uploaded files land here (runtime)
└── logs/qa_history.txt       ← Every Q&A is logged here
```

04 Prerequisites — Install These First

Before writing any code, make sure these are installed on your machine.

4.1 Python 3.9+

```
# Check version
python3 --version

# Install on Ubuntu if missing
sudo apt update && sudo apt install python3 python3-pip python3-venv -y
```

4.2 Tesseract OCR

```
# Ubuntu / Debian
sudo apt install tesseract-ocr poppler-utils -y

# macOS
brew install tesseract poppler

# Verify
tesseract --version
```

4.3 Docker

```
# Ubuntu
sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker

# Verify
docker --version
```

⚠ **Windows Users**

Install Tesseract from github.com/UB-Mannheim/tesseract/wiki and add C:\Program Files\Tesseract-OCR to your system PATH. Install Docker Desktop from docker.com.

05 Step-by-Step: Build the Application

Step 1 — Create Project & Virtual Environment

Set up the folder structure and isolate dependencies.

```
mkdir ask-the-docs && cd ask-the-docs
mkdir -p utils data/uploads logs

# Create & activate virtual environment
python3 -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
```

Step 2 — Install Dependencies

Download all required Python libraries (~2 GB total).

Create a file called requirements.txt with this exact content:

```
streamlit==1.28.1
PyPDF2==3.0.1
pdfplumber==0.10.3
pytesseract==0.3.10
pdf2image==1.16.3
Pillow==10.1.0
langchain==0.0.335
langchain-community==0.0.1
sentence-transformers==2.2.2
faiss-cpu==1.7.4
transformers==4.35.2
torch==2.1.1
accelerate==0.25.0

pip install -r requirements.txt
```

⏳ This Takes 5–10 Minutes

It downloads ~2 GB of libraries. Make sure you have a stable internet connection. If you see 'Visual C++ required' on Windows, install Visual Studio Build Tools first.

Step 3 — Write the Document Loader

This module reads PDF and TXT files. It auto-detects scanned PDFs and switches to OCR.

Create utils/document_loader.py:

```
import os, PyPDF2, pdfplumber, pytesseract
from pdf2image import convert_from_path

class DocumentLoader:
    def load_pdf(self, path, use_ocr=False):
        text = self._extract_text_pypdf(path)
        if len(text.strip()) < 100:          # Likely scanned
            text = self._extract_text_pdfplumber(path)
```

```
        if len(text.strip()) < 100 or use_ocr:
            text = self._extract_text_ocr(path)
        return text

    def _extract_text_ocr(self, path):
        images = convert_from_path(path, dpi=300)
        return '\n'.join(
            pytesseract.image_to_string(img) for img in images
        )
```

Step 4 — Write the Text Chunker

Splits large documents into overlapping pieces so no context is lost.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

class TextProcessor:
    def __init__(self, chunk_size=500, chunk_overlap=50):
        self.splitter = RecursiveCharacterTextSplitter(
            chunk_size=chunk_size,
            chunk_overlap=chunk_overlap,
            separators=['\n\n', '\n', '. ', ' ', '']
        )

    def chunk_text(self, text):
        return [c.strip() for c in self.splitter.split_text(text) if c.strip()]
```

Step 5 — Write the Embedding Manager

Converts chunks to vectors and builds the FAISS search index.

```
import numpy as np, faiss
from sentence_transformers import SentenceTransformer

class EmbeddingManager:
    def __init__(self):
        self.model = SentenceTransformer('all-MiniLM-L6-v2')
        self.dimension = self.model.get_sentence_embedding_dimension()

    def build_index(self, chunks):
        embeddings = self.model.encode(chunks).astype('float32')
        self.index = faiss.IndexFlatL2(self.dimension)
        self.index.add(embeddings)
        self.chunks = chunks

    def search(self, query, top_k=3):
        q = self.model.encode([query]).astype('float32')
        distances, indices = self.index.search(q, top_k)
        return [(self.chunks[i], d) for i, d in zip(indices[0], distances[0])]
```

Step 6 — Write the LLM Handler

Takes a question + context chunks and generates an answer using FLAN-T5.

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
```

```
class LLMHandler:
    def __init__(self):
        model_name = 'google/flan-t5-base'
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

    def generate_answer(self, question, context_chunks):
        context = '\n\n'.join(context_chunks)
        prompt = f'''Context: {context}

        Answer based ONLY on the context above.
        Question: {question}
        Answer:'''

        inputs = self.tokenizer(prompt, return_tensors='pt',
                                max_length=512, truncation=True)
        out = self.model.generate(**inputs, max_length=200, num_beams=4)
        return self.tokenizer.decode(out[0], skip_special_tokens=True)
```

Step 7 — Build the Streamlit UI

The main app.py ties every module together into a web interface.

```
import streamlit as st
from utils.document_loader import DocumentLoader
from utils.text_processor import TextProcessor
from utils.embeddings import EmbeddingManager
from utils.llm_handler import LLMHandler

st.title('📚 Ask The Docs')

uploaded = st.sidebar.file_uploader('Upload PDF/TXT', type=['pdf','txt'])
use_ocr = st.sidebar.checkbox('Enable OCR (scanned PDFs)')

if uploaded and st.sidebar.button('Process'):
    # Save → Load → Chunk → Embed → Index
    # (See full code in GitHub repo)

question = st.text_input('Ask a question:')
if question:
    chunks = embedding_manager.search(question, top_k=3)
    answer = llm_handler.generate_answer(question, chunks)
    st.success(answer)
```

06 Run & Test Locally

```
# Make sure venv is active
source venv/bin/activate

# Launch the app
streamlit run app.py

# Opens automatically at:
# http://localhost:8501
```

Testing Checklist

1. Upload a normal text-based PDF → ask a question about it
2. Upload a scanned PDF → enable OCR → ask a question
3. Ask a question that CANNOT be answered from the doc → verify it says so
4. Check logs/ folder → qa_history.txt should have entries

07 Dockerize the Application

Docker packages your entire app — code, libraries, system tools — into a single portable box. It runs identically on your machine, a friend's machine, or a cloud server.

7.1 The Dockerfile

```
FROM python:3.9-slim

WORKDIR /app

# Install OCR + PDF tools at the system level
RUN apt-get update && apt-get install -y \
    tesseract-ocr poppler-utils libgomp1 && \
    rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
RUN mkdir -p data/uploads logs

EXPOSE 8501

CMD ["streamlit", "run", "app.py",
     "--server.address", "0.0.0.0"]
```

7.2 Build & Run

```
# Build the image (~5-10 min, downloads layers)
docker build -t ask-the-docs:latest .

# Run it
docker run -p 8501:8501 ask-the-docs:latest

# Visit http://localhost:8501
```

Troubleshooting Docker

If port 8501 is busy: use docker run -p 8502:8501 ... and visit :8502. If 'no space left': run docker system prune -a to clean up old images.

08 Deploy to AWS EC2

8.1 Launch an EC2 Instance

5. Go to aws.amazon.com → sign in → search EC2 → Launch Instance
6. AMI: Ubuntu Server 22.04 LTS (Free tier eligible)
7. Instance type: t2.medium (4 GB RAM — required for the models)
8. Create a key pair (.pem file) — save it, you cannot re-download
9. Security group: Allow SSH (22), HTTP (80), and Custom TCP (8501) from 0.0.0.0/0
10. Storage: 30 GB gp3 (default). Click Launch.

8.2 Connect & Install Docker

```
# SSH into your instance (replace IP with yours)
ssh -i ~/your-key.pem ubuntu@YOUR_EC2_IP

# Install Docker
sudo apt update
sudo apt install docker.io -y
sudo systemctl start docker && sudo systemctl enable docker
sudo usermod -aG docker ubuntu

# Log out and back in
exit
```

8.3 Transfer & Build

```
# On your LOCAL machine — package the project
cd ask-the-docs
tar -czf ask-the-docs.tar.gz .
scp -i ~/your-key.pem ask-the-docs.tar.gz ubuntu@YOUR_EC2_IP:~

# On EC2 — extract and build
ssh -i ~/your-key.pem ubuntu@YOUR_EC2_IP
mkdir ask-the-docs && tar -xzf ask-the-docs.tar.gz -C ask-the-docs/
cd ask-the-docs
docker build -t ask-the-docs:latest .
```

8.4 Run with Auto-Restart

```
docker run -d \
  --name ask-the-docs-app \
  --restart=always \
  --memory='3.5g' \
  -p 8501:8501 \
  ask-the-docs:latest

# Wait ~60 seconds, then test:
```

```
curl http://localhost:8501
```

🎉 **Your App Is Live!**

Open a browser and go to http://YOUR_EC2_IP:8501 — share this URL with anyone. The app runs 24/7 independently of your laptop.

8.5 Save Your Docker Image Locally (Do This Before Stopping)

Before you stop the EC2 instance, save the Docker image to your laptop. This way you never need to rebuild from scratch. Redeployment becomes a 30-second job.

```
# On EC2 — export the image as a file
docker save ask-the-docs:latest -o ~/ask-the-docs-image.tar

# On your LOCAL machine — download it from EC2
scp -i ~/your-key.pem ubuntu@YOUR_EC2_IP:~/ask-the-docs-image.tar ~/

# Later, to load it back on ANY machine:
docker load -i ~/ask-the-docs-image.tar

# Then just run it — no rebuild needed:
docker run -d --restart=always --memory='3.5g' -p 8501:8501 --name ask-the-docs-
app ask-the-docs:latest
```

8.6 Stop the Instance (Save Money)

EC2 charges by the hour. Once you have tested your app and saved the Docker image locally, you can stop the instance. The app goes offline, but your GitHub repo and guide keep all the value. Anyone following this guide can build and deploy their own instance.

```
# Option A: Stop from AWS Console
# Go to EC2 → Select Instance → Instance State → Stop Instance

# Option B: Stop from CLI
aws ec2 stop-instances --instance-ids i-XXXXXXXXXXXXXXXXX --region YOUR_REGION
```

💡 **Redeploy Anytime in Under 2 Minutes**

To bring it back: (1) Start the EC2 instance from AWS Console, (2) SSH in, (3) docker load -i the saved .tar file, (4) docker run. Done. Or launch a brand new instance and repeat steps 8.2–8.4. The entire process takes less than 2 minutes if the image is already saved.

09 AWS Cost Breakdown — Be Transparent

This is the honest cost breakdown. The AI models and all libraries are 100% free. The only cost is the cloud server that hosts the app.

Service	Free Tier	Demo Window
EC2 t2.medium (Recommended)	✗ Not Free Tier	~$3 – $4 (3-day demo)
EC2 t2.micro (Free Tier)	✓ 750 hrs/month	$0 — but will crash (1 GB RAM)
EBS Storage (30 GB)	✓ 30 GB free	~$2.40 / month
Data Transfer (out)	✓ 1 GB free	Minimal

💰 The Smart Strategy: Stop When Done

EC2 charges by the hour. The recommended approach: start the instance before your demo, test it, share the URL, then stop it when done. Total cost for a 3-day window is around $3–$4. Save your Docker image locally first (see Step 8.5) so you can redeploy in under 2 minutes anytime.

10 Common Issues & Fixes

✕ Exit Code 137 — Container Killed

This means Out of Memory. The container used more RAM than available. Fix: use --memory='3.5g' flag when running, or switch to flan-t5-small (uses ~400 MB less).

✕ 'This site can't be reached'

The container likely stopped. SSH in, run docker ps -a to check status. If 'Exited', remove and re-run with --restart=always. Also verify port 8501 is open in your Security Group.

✕ OCR Gives Garbled Text

Increase DPI to 400 in convert_from_path(). Pre-process images (increase contrast). For critical documents, consider cloud OCR services like AWS Textract.

✕ Slow Answers (5+ seconds)

Normal on CPU. Switch to flan-t5-small for ~2x speed. In production, consider a GPU instance or caching frequent answers.

11 Limitations — Know These

Being transparent about what this app cannot do is a sign of engineering maturity. Here is the honest list:

- **Single user, single document —** No multi-user session management. Each browser session is independent.
- **CPU inference is slow —** Answers take 2–5 seconds. A GPU would cut this to <1 second.
- **OCR accuracy varies —** Poor quality scans may produce errors. Manual correction may be needed.
- **No conversational memory —** Each question is independent. Follow-up questions don't have context from previous ones.
- **Model quality —** FLAN-T5-base is good for factual extraction but struggles with complex reasoning. Larger models would improve this.

12 Final Notes & Next Steps

You now have a fully working, Dockerized, cloud-deployed RAG application. The live EC2 instance can be stopped to save money — the real value lives in your GitHub repo and this guide. Anyone who follows these steps can build and deploy their own version in under 2 hours.

What You Actually Delivered

- **A working RAG pipeline —** from PDF upload to AI-generated answers, built from scratch.
- **A Dockerized application —** runs identically on any machine. Zero environment issues.
- **A live deployment —** was publicly accessible on AWS. Can be redeployed in under 2 minutes using the saved Docker image.
- **A complete guide —** this document. Reproducible by anyone. That is the lasting value.

What You Can Build Next

- Add conversational memory (keep context across questions)
- Support multiple document uploads with metadata filtering
- Swap FAISS for a persistent vector DB (Pinecone, Weaviate)
- Deploy on GPU for 10x faster inference
- Add user authentication and rate limiting
- Set up CI/CD pipeline with GitHub Actions

🚀 **You built a real AI product.**

The live app proved it works. The guide proves anyone can build it.

That is the real deliverable.